



International University Of Business Agriculture And Technology

Lab Report:

Enumeration Constructor and Access modifier.

SUBMITTED TO:

Jabunnesa Jahan Sara

Senior Lecturer

Department of Computer Science and Engineering

SUBMITTED BY:

Fatema Taj Mim

ID: 23103406

COURSE CODE: CSC 440

SECTION: C

Date Of Submission : 22/10/2025

Problem Statement :

1. Write a C# program that Create an enumeration Days Of Week to represent the days of the week (Monday to Sunday). Then write a C# program that prompts the user to enter a number (1 to 7) and displays the corresponding day of the week using the Enumeration using System.

Objectives:

The objective of this problem is to demonstrate the use of **enumerations (enums)** in C#. It aims to show how to define a set of named constants and use them to represent fixed values — in this case, the days of the week. Understand how to **declare and use enums** in C#. Convert user input (numeric values) into corresponding Enum members. Improve **code readability** by replacing numeric codes with meaningful names. Practice **input validation** and conditional control structures.

Used Technologies and Tools

- **Programming Language:** C#
- **IDE:** Visual Studio / Visual Studio Code
- **Framework:** .NET SDK
- **Compiler:** C# Compiler (csc.exe)
- **Operating System:** Windows

Implementation Steps:

Step 1: Start Visual Studio or any C# compiler.

Step 2: Create a new Console Application project.

Step 3: Write the respective C# code for each problem.

Step 4: Compile and run each program.

Step 5: Test with different inputs to verify correctness.

Step 6: Take screenshots of the output for the report

Source Code and Output:

```
using System;

namespace EnumExample1
{
    enum DaysOfWeek
    {
        Monday = 1,
        Tuesday,
        Wednesday,
        Thursday,
        Friday,
        Saturday,
        Sunday
    }

    class Program
    {
        static void Main()
        {
            Console.Write("Enter a number (1 to 7): ");

            int dayNumber = Convert.ToInt32(Console.ReadLine());

            if (dayNumber >= 1 && dayNumber <= 7)
            {
```

```
DaysOfWeek day = (DaysOfWeek)dayNumber;

Console.WriteLine($"The day is: {day}");

}

else

{

    Console.WriteLine("Invalid input! Please enter a number between 1 and 7.");

}

}

}

}
```

Output:

Output Clear

Enter a number (1 to 7): 3
The day is: Wednesday

=== Code Execution Successful ===

Problem Statement :

2. Write a C# program that Create an enumeration Traffic Light with values Red, Yellow, and Green. Write a C# program that simulates a simple traffic light system where based on the current value of Traffic Light, the program prints different messages for each light color.

Objectives:

The objective of this problem is to simulate a **simple traffic light system** using an enumeration. It teaches how to use Enums to control program logic and output based on defined constant values. Learn how to **define and access enum values** in a switch statement. Understand how enums can make **decision-making logic** clearer and more structured. Practice **conditional statements (switch-case)** with enum types. Strengthen understanding of **real-world applications** of enumerations in control systems.

Used Technologies and Tools

- **Programming Language:** C#
- **IDE:** Visual Studio / Visual Studio Code
- **Framework:** .NET SDK
- **Compiler:** C# Compiler (csc.exe)
- **Operating System:** Windows

Implementation Steps:

Step 1: Start Visual Studio or any C# compiler.

Step 2: Create a new Console Application project.

Step 3: Write the respective C# code for each problem.

Step 4: Compile and run each program.

Step 5: Test with different inputs to verify correctness.

Step 6: Take screenshots of the output for the report

Source Code and Output:

```
using System;

namespace EnumExample2
{
    enum TrafficLight
    {
        Red,
        Yellow,
        Green
    }

    class Program
    {
        static void Main()
        {
            Console.Write("Enter the traffic light color (Red, Yellow, Green): ");

            string input = Console.ReadLine();

            if (Enum.TryParse(input, true, out TrafficLight light))
            {
                switch (light)
                {
                    case TrafficLight.Red:
                        Console.WriteLine("Stop! The light is Red.");
                        break;
```

```
        case TrafficLight.Yellow:
            Console.WriteLine("Get Ready! The light is Yellow.");
            break;
        case TrafficLight.Green:
            Console.WriteLine("Go! The light is Green.");
            break;
    }
}
else
{
    Console.WriteLine("Invalid color entered!");
}
}
}
```

Output:

Output

Clear

```
Enter the traffic light color (Red, Yellow, Green): Green
Go! The light is Green.

=== Code Execution Successful ===
```

Output

Clear

```
Enter the traffic light color (Red, Yellow, Green): Red
Stop! The light is Red.

=== Code Execution Successful ===
```

```
Output Clear  
Enter the traffic light color (Red, Yellow, Green): Yellow  
Get Ready! The light is Yellow.  
  
=== Code Execution Successful ===
```

Problem Statement :

3. Write C# program that Define a default constructor that initializes these fields with default values: "Unknown" for Brand, "Unknown" for Model , and 0 for Year. Write a program that creates an instance of the Car class using the default constructor and displays the values of the fields.

Objectives:

The objective of this problem is to demonstrate how to create a **class with fields and a default constructor** in C#. It helps learners understand how constructors initialize objects with default values and how class instances store and display data. Learn to **define classes** with multiple fields (data members). Understand how to **use constructors** to assign initial values. Practice **object creation** and **field access** in C#. Strengthen understanding of **object-oriented programming (OOP)** principles, particularly **encapsulation** and **initialization**.

Used Technologies and Tools

- **Programming Language:** C#
- **IDE:** Visual Studio / Visual Studio Code
- **Framework:** .NET SDK
- **Compiler:** C# Compiler (csc.exe)
- **Operating System:** Windows

Implementation Steps:

Step 1: Start Visual Studio or any C# compiler.

Step 2: Create a new Console Application project.

Step 3: Write the respective C# code for each problem.

Step 4: Compile and run each program.

Step 5: Test with different inputs to verify correctness.

Step 6: Take screenshots of the output for the report

Source Code and Output:

```
using System;

namespace CarExample
{
    class Car
    {
        public string Brand;
        public string Model;
        public int Year;

        // Default constructor
        public Car()
        {
            Brand = "Unknown";
            Model = "Unknown";
            Year = 0;
        }
    }

    class Program
    {
        static void Main()
        {
```

```
Car myCar = new Car();

Console.WriteLine("Car Information:");

Console.WriteLine($"Brand: {myCar.Brand}");

Console.WriteLine($"Model: {myCar.Model}");

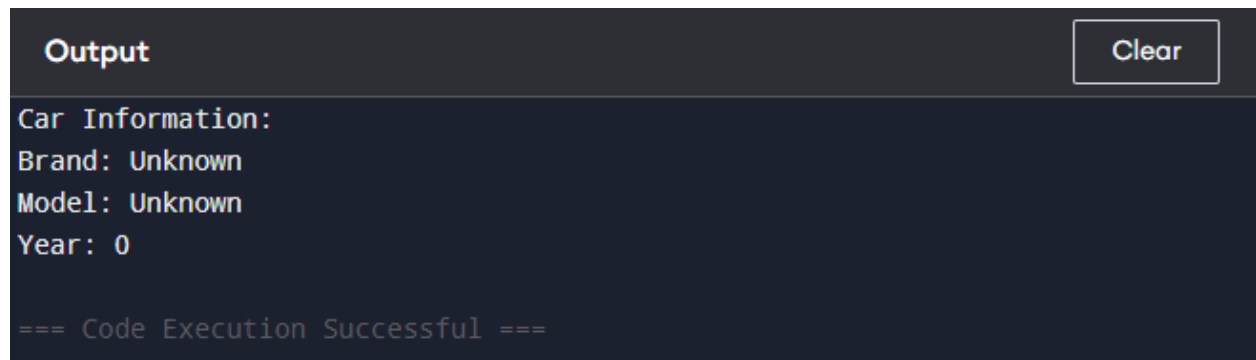
Console.WriteLine($"Year: {myCar.Year}");

    }

}

}
```

Output:



```
Output
Car Information:
Brand: Unknown
Model: Unknown
Year: 0

=== Code Execution Successful ===
```

Problem Statement :

4. Write a c# program that Access Modifiers

Objectives:

The objective of this program is to **demonstrate the concept and use of access modifiers in C#**. Access modifiers define the **visibility and accessibility** of class members (fields, methods, and properties) within the program or across other classes and assemblies. Explain the role of access modifiers in C#. Choose the correct access modifier for different programming scenarios. Understand how inheritance affects accessibility of class members. Write more **secure and maintainable** object-oriented code.

Used Technologies and Tools

- **Programming Language:** C#
- **IDE:** Visual Studio / Visual Studio Code
- **Framework:** .NET SDK
- **Compiler:** C# Compiler (csc.exe)
- **Operating System:** Windows

Implementation Steps:

Step 1: Start Visual Studio or any C# compiler.

Step 2: Create a new Console Application project.

Step 3: Write the respective C# code for each problem.

Step 4: Compile and run each program.

Step 5: Test with different inputs to verify correctness.

Step 6: Take screenshots of the output for the report

Source Code and Output:

```
using System;

namespace AccessModifiersExample
{
    public class Person
    {
        public string Name;
        private int age;
        protected string Address;
        private protected string SecretCode;
        public Person(string name, int age, string address, string secretCode)
        {
```

```
        Name = name;

        this.age = age;

        Address = address;

        SecretCode = secretCode;
    }

    public int GetAge()
    {
        return age;
    }
}

public class Employee : Person
{
    public Employee(string name, int age, string address, string secretCode)
        : base(name, age, address, secretCode)
    {
    }

    public void ShowDetails()
    {
        Console.WriteLine($"Name: {Name}");

        Console.WriteLine($"Address: {Address}");

        Console.WriteLine($"Secret Code: {SecretCode}");
    }
}
```

```
}  
  
class Program  
{  
    static void Main()  
    {  
        Person person = new Person("Mac ", 25, "Gulshan", "XYZ123");  
  
        Console.WriteLine($"Name: {person.Name}");  
  
        Console.WriteLine($"Age: {person.GetAge()}");  
  
  
        Employee employee = new Employee("Boby", 28, "456 Avenue", "ABC789");  
  
        employee.ShowDetails();  
  
  
        Console.ReadLine();  
    }  
}  
}
```

Output:

OutputClear

```
Name: Mac  
Age: 25  
Name: Boby  
Address: 456 Avenue  
Secret Code: ABC789
```